Programação Orientada a Objetos

Departamento de Computação
Universidade Federal de Sergipe

Polimorfismo

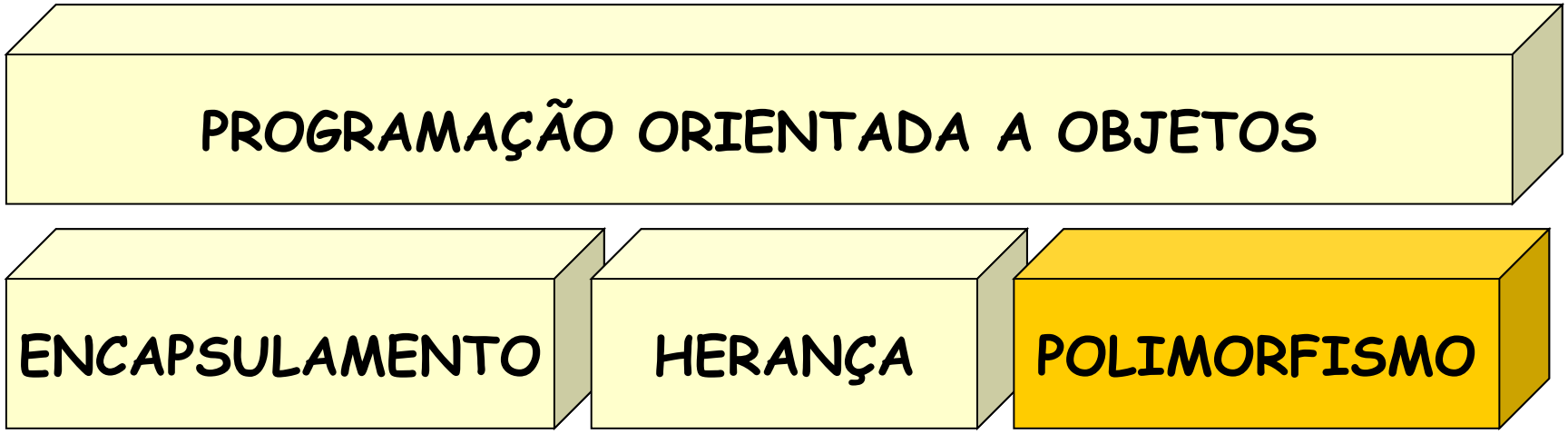
Prof. Kalil Araujo Bispo
kalil@dcomp.ufs.br

Roteiro

- Pilares da Programação OO
- Vinculação Estática
- Vinculação Dinâmica
- Sobrecarga de Operações
- Polimorfismo

Pilares da Programação OO

- **O que vamos estudar hoje?**

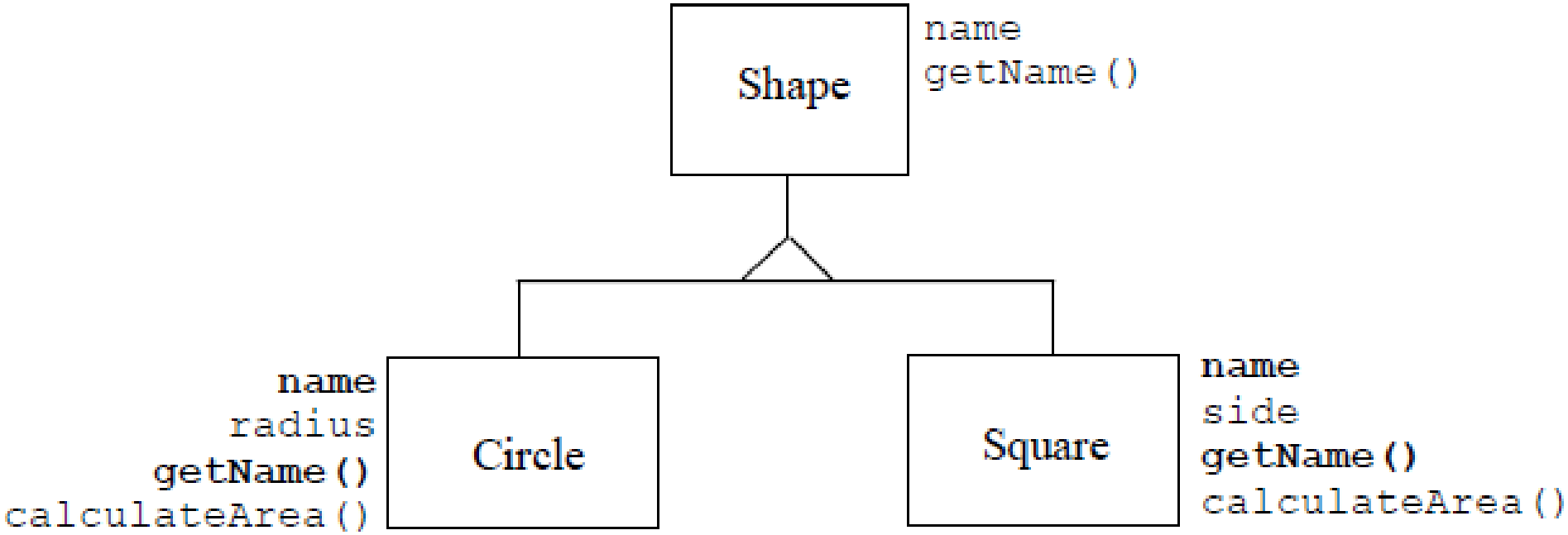


Vinculação Estática

- **Vinculação**
  - Associação entre entidades de um programa e seus atributos

- **Vinculação estática**
  - Ocorre durante a compilação do programa

Vinculação Estática



Vinculação Estática

```
Data Section
Type
 Circle = Record
        String name;
        int radius;
      End
 Square = Record
        String name;
        int side;
          End
Variable
 shapeArray : Array [1..5] of
 char;
 c : Circle;
 s : Square;
Main Code Section
 c.name = "Circle C";
 c.radius = 2;
 s.name = "Square S";
 s.side  = 3;
 shapeArray[1] = 'c';
 shapeArray[2] = 's';
```

```
For int i = 1 to 2 do {
 Switch shapeArray[i]
     'c' : calculateCircleArea();
     's' : calculateSquareArea();
     'n' : do nothing;
 End (Case)
}

Routine Section
calculateCircleArea(){
  float area = 3.14 * c.radius *
c.radius;
  writeln ("The area of ", c.name, "
is ", area, " sq. cm.");
}

calculateSquareArea() {
   float area = s.side * s.side;
   writeln ("The area of ",s.name, "
is “, area, " sq. cm.");
}
```

Vinculação Estática

- **Uso de case para decidir método a ser chamado.**
- **Rotinas de desenho com nomes diferentes.**
- **Variável shapeArray[i] é vinculada estaticamente às rotinas de desenho através de seu valor (‘c’ ou ‘s’).**
- **Problemas na manutenção de código**
  - Exemplo: adição da forma triângulo ao programa.

Vinculação Estática

```
Data Section
Type
   ...
   Triangle = Record
     String name;
     int base, height;
   End

Variable
...
t : Triangle;


Main Code Section
  ...
  t.name = "Triangle T";
  t.base = 4;
  t.height = 5;
  ...
A[3] = 't';
```

```
For int i = 1 to 3 do {
  Switch shapeArray[i]
   'c' : calculateCircleArea();
   's' : calculateSquareArea();
   't' : calculateTriangleArea();
   'n' : do nothing;
  End (Case)
}

Routine Section
calculateCircleArea()  {
    ...
}
calculateSquareArea()  {
    ...
}
calculateTriangleArea(){
   float area = 0.5f * t.base *
        t.height;
   writeln ("The area of ", t.name,
" is ", area, " sq. cm.");
}
```

Vinculação Dinâmica

- **Vinculação de variáveis a rotinas ou métodos é feita em tempo de execução**

Vinculação Dinâmica

```
class Shape {
   private String name;
   Shape(String aName){
     Name = aName;
   }
   public String getName() {
     return name;
   }
   public float calculateArea(){
     return 0.0f;
   }

   public static void main(String argv[]) {
      Circle c = new Circle(”Circle C”);
      Square s = new Square(”Square S”);
      Shape shapeArray[] = {c, s};
      for (int i=0; i<shapeArray.length; i++) {
         System.out.println("The area of " + shapeArray[i].getName() +
                  “ is " + shapeArray[i].calculateArea()+" sq. cm.\n");
      }
   }
}
```

Vinculação Dinâmica

```
class Circle extends Shape {

   private int radius;

   Circle(String aName) {
      super(aName);
      radius = 3;
   }

   public float calculateArea(){
      float area;
      area = (float) (3.14 *
              radius * radius);
      return area;
   }
}
```

```
class Square extends Shape {

   private int side;

   Square(String aName) {
      super(aName);
      side = 3;
   }

   public float calculateArea() {
      int area;
      area = side * side;
      return area;
   }
}
```

Vinculação Dinâmica

- O método é selecionado com base na classe da forma referenciada em shapeArray em tempo de execução

Sobrecarga de Operações

- **Métodos que possuem a mesma assinatura, mas têm implementações diferentes**
  - calculateArea()

- **É dito que dois métodos possuem a mesma assinatura quando:**
  - O nome dos métodos é o mesmo.
  - O número e tipo dos parâmetros formais é o mesmo.

Sobrecarga de Operações

- **Sobrecarga de Nomes de Métodos**
  - Métodos que possuem a mesma assinatura podem gerar problemas na compilação, a depender de onde são usados no programa

Sobrecarga de Operações

```
class A {
   ...
   A() { ... }
   A(int x) { ... }
   A(int x, int y) { ... }
   A(String x) { ... }
   ...
}
```

`A thisA = new A();`	→	`A()`
`A thisA = new A(3);`	→	`A(int x)`
`A thisA = new A(4, 5);`	→	`A(int x, int y)`
`A thisA = new A("hello");`	→	`A(String x)`

Sobrecarga de Operações

```
class A {
 A() {}
 public void a1() {}
 public void a1() {}
}
```

Sobrecarga de Operações

```
class A {
  A() {}
  public void a1() {}

  public static void main(String args[]) {
  }
}

class B {

  B() {}
  public void a1() {}
  public void b1() {}
}
```

Sobrecarga de Operações

```
class C {
  C() {}
  public void c1() {System.out.println("C.c1()");}
}

class D extends C {
  D() {}
  public void c1() {
    super.c1();
    System.out.println("D.c1()");
  }
  public void d1() {}

  public static void main(String args[]) {
    D thisD = new D();
    thisD.c1();
  }
}
```

Sobrecarga de Operações

```
class C {
  C() {}
  public void c1() {System.out.println("C.c1()");}
}

class D extends C {
  D() {}
  public void c1() {
    super.c1();
    System.out.println("D.c1()");
  }
  public void d1() {}

  public static void main(String args[]) {
    D thisD = new D();
    thisD.c1();
  }
}
```

```
C.c1()
D.d1()
```

Sobrecarga de Operações

- **O método c1 da subclasse D redefine o método c1 da superclasse C**
  - Assinatura deve ser a mesma

Sobrecarga de Operações

```
class C {
  C() {}
  public void c1() {System.out.println("C.c1()");}
}

class D extends C {
  D() {}
  public void c1(int i) {
    super.c1();
    System.out.println("D.c1()");
  }
  public void d1() {}

  public static void main(String args[]) {
    D thisD = new D();
    thisD.c1();
    thisD.c1(3);
  }
}
```

Sobrecarga de Operações

```
class C {
  C() {}
  public void c1() {System.out.println("C.c1()");}
}

class D extends C {
  D() {}
  public void c1(int i) {
    super.c1();
    System.out.println("D.c1()");
  }
  public void d1() {}

  public static void main(String args[]) {
    D thisD = new D();
    thisD.c1();
    thisD.c1(3);
  }
}
```

```
C.c1()
C.c1()
D.c1()
```

Polimorfismo

- **A vinculação dinâmica permite a chamada apropriada de métodos sem a necessidade de referências diretas na mensagem**
  - calculateArea(): ao ser enviada do main(), a mensagem é interpretada de acordo pelo receptor (circulo, quadrado...)
- **Polimorfismo em POO: habilidade de diferentes objetos executarem respostas apropriadas a uma mesma mensagem**

Polimorfismo

- **Seleção do Método**
  - A interpretação da mensagem não é afetada pelo objeto emissor
  - O emissor não sabe que objeto da classe responderá a mensagem, nem como ele executará o método
  - A seleção do método apropriado depende da classe à qual o objeto receptor pertence
  - Exemplo: o método calculateArea() é polimórfico

Polimorfismo

- **Desenvolvimento Incremental**
  - Polimorfismo é possibilitado pela vinculação dinâmica e pela habilidade de se usar o mesmo nome para métodos similares através de definições de classes.
  - Encoraja o programador a especificar que método será usado, ao invés de focar no que ele faz
  - Exemplo: adição de triângulo

Polimorfismo

```
class Triangle extends Shape {
  private int base, height;

  Triangle(String aName) {
    super(aName);
    base   = 4;  height = 5;
  }

  public float calculateArea() {
    float area = 0.5f * base * height;
    return area;
  }
}
```

Polimorfismo

```
class Shape {
  ...
  public static void main(String argv[]) {
    Circle c = new Circle("Circle C");
    Square s = new Square("Square S");
    Triangle t = new Triangle("Triangle T");
    Shape shapeArray[] = {c, s, t};
    for (int i=0; i<shapeArray.length; i++) {
      System.out.println("The area of " + shapeArray[i].getName()
           + " is " + shapeArray[i].calculateArea()+" sq. cm.\n");
    }
  }
}
class Circle extends Shape {
...
}
class Square extends Shape {
 ...
}
```

Polimorfismo

- **Melhora na legibilidade do código**
  - A mesma mensagem é usada para chamar diferentes objetos
  - Simplificação do código

Referências

- Poo, Kiong e Ashok. Object Oriented Programming and Java. Segunda edição. 2008.
- Slides "Pilares da POO - Polimorfismo" Profa. Beatriz Trinchão. UFS. 2013.

Programação Orientada a Objetos

Departamento de Computação
Universidade Federal de Sergipe

Polimorfismo

Prof. Kalil Araujo Bispo
kalil@dcomp.ufs.br